

코딩테스트 추가 학습

기준 5일차 이후 · C++ 문제 해결과 실전 재풀이

현재 자료 다음에, 무엇을 직접 해볼까

이미 작성된 배열·해시·정렬·탐색·DP 노트를 발판으로, 새 유형을 정확히 선택하고 반례로 검증하는 연습을 합니다. 자작 예제의 답과 공식 확장 문제, 6주 실행 계획을 함께 담았습니다.

기준 폴더	코테준비
검토 자료	Markdown 6개
기준 직무	C++ 개발 직무 이직을 준비하는 학습자
권장 활용	주 6-8시간을 이 폴더에 배정하는 6주 집중 과정

읽기 → 구현·실험 → 결과 설명

자료 보유와 학습 완료는 별개입니다. 각 장의 통과 기준으로 시작 위치를 정하세요. 세 폴더의 계획은 개별 집중 기준이며, 병행할 때는 총 가용 시간에 맞춰 기간을 늘립니다.

01 / 기존 자료에서 이어가기

5일차 다음은 유형 선택과 검증

새 알고리즘과 기존 풀이의 완성도를 함께 높이는 후속 실습입니다.

기존 1~3일차는 배열·문자열·해시·스택·큐·우선순위 큐를 다룹니다. 4일차는 정렬뿐 아니라 정답 이분 탐색과 LIS까지, 5일차는 DFS/BFS·다중 시작점 BFS·백트래킹·배낭·LCS까지 포함합니다. 파일 이름만 보고 이 주제를 전부 새 학습으로 배정하지 않았습니다.

구분	추가 진행
새로 보강	다수 구간 질의의 누적합, 구간 갱신 차분, 슬라이딩 윈도우, 가중 최단거리, DSU
기존 개념 심화	정답 이분 탐색의 경계·단조성, DP 갱신 순서, 탐욕법의 교환 논증
실전 적용	제약에서 복잡도 결정 → 불변식 설명 → 반례 검사 → 시간 제한 재풀이

이 폴더의 목적은 이직 과정에서 C++ 문제 해결을 정확히 보여주는 것입니다. 실제 지원 기업의 시험 유형·언어·시간이 확인되면 마지막 모의고사를 그 형식에 맞추습니다. 현재 자료에서 확인되지 않는 주제가 실제 미경험을 뜻하지는 않습니다.

시작 전 40분 점검

- 기존 4일차에서 경계 탐색 1문제, 5일차에서 격자 BFS 1문제를 해설 없이 푼다.
- 풀이 전에 시간·공간 복잡도와 인덱스 범위를 말한다. 실패한 항목만 해당 노트로 돌아간다.
- 두 문제를 풀고 방문 처리 시점·lower_bound 반환 의미를 설명할 수 있으면 이번 과정으로 진행한다.

공부 방식

본문의 자작 작은 예제를 먼저 손으로 계산하고, 구현한 뒤 공식 문제로 확장합니다. 하루 번호는 기존 5일차 뒤의 학습 단위이며 하루 안에 모든 단원을 끝내라는 뜻이 아닙니다.

02 / 6일차 · 누적합과 차분

구간마다 다시 계산하지 않기

기존 배열·빈도 배열 지식을 다수 질의와 대량 갱신으로 확장합니다.

길이 N 배열에 구간 합 질의 Q 개를 각각 순회하면 최악 $O(NQ)$ 입니다. $P[0]=0$, $P[i+1]=P[i]+a[i]$ 로 만들면 반 열린 구간 $[l,r)$ 의 합은 $P[r]-P[l]$ 입니다. 전처리 $O(N)$, 질의 $O(1)$, 공간 $O(N)$. 이후 배열 값이 바뀌지 않는 상황에 적합합니다.

```
a = [3, -2, 5, 1]
P = [0, 3, 1, 6, 7]
sum([1, 4)) = P[4] - P[1] = 4
sum([0, 0)) = 0

// Range add on inclusive [l, r]:
diff[l] += delta;
diff[r + 1] -= delta;
// Prefix-sum diff once after ALL updates.
```

차분 배열은 변화가 시작되는 지점과 끝나는 다음 지점만 기록합니다. 길이 $N+1$ 로 잡아 $r=N-1$ 의 다음 칸도 기록합니다. 갱신마다 현재 값이 즉시 필요한 문제에서는 마지막 한 번의 복원만으로 충분하지 않습니다. 질의 순서와 온라인/일괄 처리 조건을 먼저 확인합니다.

자작 실습과 정답

- $N=5$, 초기값 모두 0. 닫힌 구간 $[1,3]$ 에 +2, $[2,4]$ 에 -1을 적용하라. 최종 배열은 $[0,2,1,1,-1]$.
- 위 배열에서 $[1,4)$ 의 합은 4. 전체 구간·빈 구간·첫 칸·끝 칸을 각각 테스트한다.
- $N=200,000$ 이고 원소 최대 10^9 라면 누적합은 2×10^{14} 까지 커진다. 입력은 int여도 합과 차분 계산의 중간값은 long long으로 둔다.

2차원은 사각형의 네 모서리에 $+v, -v, -v, +v$ 를 기록하고 행·열 누적합으로 복원합니다. 먼저 2×3 격자 전체에 +2를 적용해 여섯 칸 모두 2가 되는지 확인한 뒤 공식 문제 '파괴되지 않은 건물'로 확장합니다.[1]

통과 기준

부호가 다른 두 구간 갱신을 겹쳐도 직접 순회한 정답과 같아야 합니다. 새 코드와 단순 $O(NQ)$ 구현을 작은 랜덤 입력으로 비교하면 경계 오류를 찾기 쉽습니다.

03 / 7일차 · 슬라이딩 윈도우

포인터가 돌아가지 않는 이유

양수 배열의 구간 합 조건에서 매번 합을 다시 구하지 않습니다.

자작 문제: 양수 배열에서 합이 K 이상인 가장 짧은 연속 구간의 길이를 구합니다. 없으면 0입니다. 오른쪽을 늘리면 합이 증가하고, 왼쪽을 빼면 감소합니다. 합이 충분한 동안 후보 길이를 기록한 뒤 왼쪽을 줄입니다. 각 원소가 한 번 들어오고 한 번 나가므로 $O(N)$, 추가 공간 $O(1)$ 입니다.

```
#include <algorithm>
#include <vector>
// Contract: every a[i] > 0, k > 0, size fits int.
int shortest(const std::vector<int>& a, long long k) {
    int n = static_cast<int>(a.size());
    int left = 0, best = n + 1;
    long long sum = 0;
    for (int right = 0; right < n; ++right) {
        sum += a[right];
        while (left <= right && sum >= k) {
            best = std::min(best, right - left + 1);
            sum -= a[left++];
        }
    }
    return best == n + 1 ? 0 : best;
}
```

- [2,1,3,2,4], $K=6 \rightarrow 2$. [2,4]가 정답 구간이다. [8], $K=6 \rightarrow 1$; [1,2], $K=6 \rightarrow 0$.
- 음수를 허용하면 [1,-1,5], $K=5$ 에서 위 방식은 길이 3을 기록한 뒤 멈춰 정답 1을 놓친다. 양수라는 전제가 핵심이다.

공식 문제 '연속된 부분 수열의 합'은 합이 정확히 K 인 구간 중 최단, 동률이면 앞 구간을 선택합니다.[2] 위 함수와 반환값 조건이 다르므로 그대로 제출하지 않습니다. 양수 조건에서 합이 K 를 초과할 때 줄이고, 같을 때 길이와 시작 인덱스를 비교하도록 바꿔 보세요.

통과 기준

합·left·right를 매 반복 적어 구간 불변식을 설명합니다. 공식 문제에서는 동률과 한 원소 정답까지 처리하고, 정렬 여부보다 양수 전제가 윈도우의 단조성에 왜 중요한지 말할 수 있어야 합니다.

04 / 7일차 확장 · 누적합과 해시

음수가 있으면 이전 합을 센다

기존 해시 빈도표를 누적합과 결합해 연속 구간의 개수를 구합니다.

자작 문제: 정수 배열에서 합이 K인 비어 있지 않은 연속 구간 수를 구합니다. 현재 누적합이 S일 때 이전 누적합이 S-K인 위치마다 정답 구간 하나가 생깁니다. freq에 이전 누적합의 출현 횟수를 저장합니다. 같은 합이 여러 번 등장할 수 있으므로 존재 여부만 저장해서는 안 됩니다.

```
#include <unordered_map>
#include <vector>
long long count_sum(const std::vector<int>& a, long long k) {
    std::unordered_map<long long, long long> freq;
    freq[0] = 1;
    long long sum = 0, answer = 0;
    for (int x : a) {
        sum += x;
        auto it = freq.find(sum - k);
        if (it != freq.end()) answer += it->second;
        ++freq[sum];
    }
    return answer;
}
```

조회한 뒤 현재 합을 저장하는 순서가 중요합니다. 반대로 하면 K=0일 때 길이 0인 구간을 세게 됩니다. freq[0]=1은 배열 시작 전의 빈 prefix이며, 시작점이 0인 실제 구간을 세기 위해 필요합니다. 해시 평균 시간 O(N), 공간 O(N)이고 충돌 최악 상황까지 O(N)을 보장하는 것은 아닙니다.

입력 / K	기대 결과와 이유
[1,-1,3,0] / 3	4개: [0,2], [0,3], [2,2], [2,3] (닫힌 인덱스)
[0,0,0] / 0	6개 = $N(N+1)/2$
[] / 0	0개: 빈 구간은 세지 않는다

통과 기준

작은 배열을 모든 [l,r] 조합으로 검사한 정답과 대조합니다. 결과 개수도 $N(N+1)/2$ 까지 커질 수 있으므로 long long을 사용합니다. 누적합 자체가 long long 범위를 넘지 않는 입력 제약도 확인하세요.

05 / 8일차 · 가중 최단거리

BFS에서 Dijkstra로 넘어갈 때

3일차의 priority_queue와 5일차의 그래프 표현을 연결합니다.

BFS는 같은 비용의 간선에서 간선 수 기준 최단거리를 구합니다. 가중치가 다른 그래프에서는 가장 먼저 발견한 경로가 최소 비용이라는 보장이 없습니다. Dijkstra는 모든 간선 가중치가 음수가 아닐 때 현재 최소 거리 후보를 꺼내 인접 정점을 완화합니다. 음수 간선이 있으면 이 확정 논리를 사용할 수 없습니다.

```
// Core loop, not a complete translation unit.
// Headers: <vector>, <queue>, <utility>, <functional>
// g[u]: vector<pair<int, long long>>; valid vertex ids.
// All weights >= 0; every valid distance < INF.
using Item = std::pair<long long, int>;
const long long INF = (1LL << 62);
std::vector<long long> dist(n, INF);
std::priority_queue<Item, std::vector<Item>,
                    std::greater<Item>> > pq;

dist[start] = 0;
pq.push({0, start});
while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du != dist[u]) continue;
    for (auto [v, w] : g[u]) {
        if (w >= INF - du) continue;
        if (du + w < dist[v]) {
            dist[v] = du + w;
            pq.push({dist[v], v});
        }
    }
}
```

방향 간선 0→1 비용 7, 0→2 비용 2, 2→1 비용 1이면 거리 [0,3,2]입니다. 처음 넣은 (7,1)은 나중에 오래된 후보로 버립니다. 중복 후보를 허용하는 힙 구현은 $O((V+E) \log(E+1))$ 수준의 시간과 $O(V+E)$ 공간으로 분석할 수 있습니다.

- 자작 검사: 도달 불가능 정점, 비용 0, 같은 두 정점의 서로 다른 비용 간선, 시작점 하나를 포함한다.
- 공식 문제 '배달'은 무방향 도로이며 같은 두 마을 사이 도로가 여러 개일 수 있다. 양방향 추가와 거리 K 이하 조건을 확인한다.[3]

통과 기준

BFS와의 차이를 위 3점점 반례로 설명합니다. 작은 비음수 그래프에서 전 정점 쌍의 단순 검사 결과와 비교하고, INF가 실제 최대 경로보다 충분히 큰지 입력 제약으로 확인합니다.

06 / 9일차 · DSU와 최소 신장 트리

연결 여부와 이동 비용은 다르다

정렬·그래프에서 연결 요소를 빠르게 합치는 자료구조로 확장합니다.

DSU는 정점들이 같은 집합에 속하는지 검사하고 두 집합을 합칩니다. 경로 자체나 최단거리를 구하지는 않습니다. Kruskal은 간선을 비용 순으로 보고, 서로 다른 집합을 잇는 간선만 선택합니다. 이미 같은 집합이면 사이클이 생기므로 버립니다.

```
#include <numeric>
#include <utility>
#include <vector>
struct DSU {
    std::vector<int> p, size;
    explicit DSU(int n) : p(n), size(n, 1) {
        std::iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        while (x != p[x]) {
            p[x] = p[p[x]];
            x = p[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) std::swap(a, b);
        p[b] = a; size[a] += size[b];
        return true;
    }
};
```

자작 그래프: (0,1,1), (1,2,2), (0,2,5), (2,3,3). 선택 비용은 1+2+3=6이며 정점 4개에 간선 3개가 선택됩니다. (2,3,3)을 제거하면 전체 연결이 불가능합니다. 선택 수가 N-1인지 확인하지 않으면 일부 연결의 비용을 정답으로 오인할 수 있습니다.

현재 연결 요소를 가르는 cut에서 가장 싼 간선을 선택해도 어떤 최소 신장 트리로 교체할 수 있다는 성질이 탐욕 선택의 근거입니다. 정렬 $O(E \log E)$ 가 주 비용이고, 경로 압축과 크기 기준 합치기는 DSU 연산의 상 각 비용을 낮춥니다. 공식 확장: '섬 연결하기'.[4]

통과 기준

자기 자신 합치기·중복 합치기에서 false가 나와야 합니다. 최소 신장 트리의 전체 연결 비용과 한 시작점에서의 최단거리 합이 다른 문제임을 설명합니다.

07 / 10일차 · 탐욕과 정답 이분 탐색

외운 템플릿 대신 판정식을 증명

기존 4일차의 파라메트릭 서치를 실패하지 않는 경계 설정으로 확장합니다.

자작 문제: 작업량 [4,2,3]을 순서대로 최대 2개 작업자 구간에 배정합니다. 작업은 쪼갤 수 없고 각 작업자의 합 상한 C를 최소화합니다. C=4면 [4],[2],[3]으로 3명이 필요하지만, C=5면 [4],[2,3]으로 2명입니다. 따라서 정답은 5입니다.

판정과 단조성

C를 고정하면 현재 작업자에게 들어갈 수 있는 작업을 앞에서부터 최대한 넣습니다. 이 방식은 첫 구간을 가능한 가장 멀리 보내므로 다른 유효한 첫 구간보다 남은 일이 많아지지 않습니다. 같은 논리를 반복하면 필요한 구간 수가 최소입니다. C에서 가능하면 더 큰 C에서도 가능하므로 false→true 경계를 찾습니다.

```
// Preconditions: nonempty a, a[i] > 0, workers >= 1.
// feasible(C): greedy number of groups <= workers.
long long lo = max_element_value;
long long hi = total_sum;
while (lo < hi) {
    long long mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid;
    else lo = mid + 1;
}
// lo is the minimum feasible capacity.
```

하한은 가장 큰 작업량, 상한은 총합으로 둡니다. 판정 $O(N)$, 전체 $O(N \log S)$, S는 탐색 범위의 크기입니다. 합·중간값은 long long을 사용하고, feasible 안에서 작업량 하나가 C보다 큰 경우도 명시적으로 거부하도록 구현합니다.

경계 입력	기대 정답
[4,2,3], 1명	9
[4,2,3], 3명	4
[7], 5명	7
[2,2,2,2], 3명	4

통과 기준

결과 C에서 $\text{feasible}(C)=\text{true}$, C가 하한보다 크면 $\text{feasible}(C-1)=\text{false}$ 인지 확인합니다. 작은 배열의 모든 절단 위치를 나열한 정답과 대조합니다. 정렬하면 작업 순서 조건이 바뀌므로 이 문제에서 정렬하지 않습니다.

08 / 11일차 · DP 갱신 순서 검증

한 줄의 반복문이 문제를 바꾼다

5일차의 배낭·LCS를 다시 설명하기보다 상태와 갱신 순서를 검증합니다.

상태 $dp[w]$ 가 '지금까지 처리한 물건으로 용량 w 이하에서 얻는 최대 가치'라면 0/1 배낭에서 물건 하나를 넣는 반복은 w 를 큰 쪽부터 순회해야 합니다. 그래야 $dp[w - \text{weight}]$ 가 현재 물건을 다시 사용한 상태가 되지 않습니다. 오름차순 갱신은 같은 물건을 여러 번 사용할 수 있는 문제로 바뀝니다.

```
// One item: weight=2, value=3, capacity=4.
// 0/1 knapsack (use at most once): answer = 3
for (int w = capacity; w >= weight; --w)
    dp[w] = std::max(dp[w], dp[w - weight] + value);

// Unbounded knapsack (reuse allowed): answer = 6
for (int w = weight; w <= capacity; ++w)
    dp[w] = std::max(dp[w], dp[w - weight] + value);
```

개수 DP에서도 순서를 시험한다

동전 [1,2]로 합 3을 만드는 조합은 {1,1,1}, {1,2}의 2개입니다. 순서가 다른 나열을 구하면 [1,1,1], [1,2], [2,1]의 3개입니다. 동전을 바깥 반복에 두는 방식과 합을 바깥 반복에 두는 방식이 어느 정의를 계산하는지 작은 표로 확인하세요. $dp[0]=1$ 은 아무것도 선택하지 않은 출발 방법 하나입니다.

- 자작 과제 A: 물건 (무게2,가치3), (무게3,가치4), 용량5의 0/1 최댓값은 7. 용량1이면 0.
- 자작 과제 B: 동전 [2]로 합3을 만드는 방법은 0. 합0은 빈 선택을 허용하는 정의에서 1.
- 불가능 상태를 0으로 표현하면 실제 가치 0과 섞이는 문제인지 확인한다. 정확한 무게를 채워야 하면 초기 상태를 별도로 설계한다.

제출 전에 상태 한 문장, 전이 근거, 초기값, 계산 순서, 정답 위치, 시간·공간 복잡도를 적습니다. 기존 LCS는 2차원 표부터 검증하고 메모리 축약은 그 뒤에 합니다. 최적화 전후가 작은 입력에서 같은지 먼저 비교합니다.

통과 기준

정답 코드뿐 아니라 의도적으로 갱신 방향을 바꾼 코드가 틀리는 가장 작은 입력을 제출합니다. 새 DP 문제에서는 점화식 암기보다 상태 정의에 5분을 먼저 쓰세요.

09 / 6주 실행 계획

한 유형을 두 번 풀고 넘어가기

주 6-8시간을 이 폴더에 배정하는 계획입니다. 다른 폴더와 병행하면 기간을 조정합니다.

주차	새 학습과 실습	남길 결과
1주	시작 점검 + 누적합·1D 차분	직접 순회 정답과 경계 비교
2주	양수 원도 + 음수 누적합/해시	동률·빈 입력·0·음수 반례
3주	Dijkstra + 배달[3]	BFS 반례와 중복 간선 검사
4주	DSU/Kruskal + 섬 연결하기[4]	연결 불가능·중복 합치기 검사
5주	정답 이분 탐색·DP 순서 심화	작은 완전탐색과 정답 대조
6주	2D 차분[1] 선택 + 90분 모의고사	풀이·검증·재풀이 기록

매주 같은 학습 주기

- 개념과 작은 예제 90분 → 자작 문제 구현 90분 → 공식 문제 90분 → 오답 분석과 재풀이 90분을 기본 6시간으로 둔다.
- 추가 2시간은 새 문제 개수보다 실패한 유형의 조건 변형에 사용한다.
- 처음 푼 뒤 2일과 7일 후에는 코드를 보지 않고 재풀이한다. 막힌 지점만 기록하고 같은 해설을 통째로 옮기지 않는다.

이미 잘하는 유형은 설명 5분 + 해설 없는 풀이 30분 + 경계 검사 10분을 통과하면 건너뛵니다. 아직 BFS나 기본 배열 구현이 불안정하면 새 그래프 유형을 늘리기 전에 기존 노트로 돌아갑니다. 세그먼트 트리·고급 문자열·기하 알고리즘은 목표 시험에서 필요하거나 위 단계가 안정된 뒤 추가합니다.

현실적인 진도 조정

한 주 과제를 이틀 연속 해설 없이 재현하지 못하면 다음 주 새 유형 하나를 줄입니다. 목표는 특정 문제 수나 난이도 배치가 아니라 처음 보는 조건을 기존 패턴과 구별하는 능력입니다.

10 / 실전과 이직 준비

90분 안에 설명까지 완성하기

경력 기술 면접과 코딩 시험에 각각 보여줄 증거를 만듭니다.

첫 모의고사: 자작 문제 세트

시간	과제	확인 기준
25분	길이 N 배열의 Q개 구간 합	입력 범위·long long·반열린 구간
35분	양수 구간 최단 길이 또는 가중 최단거리	조건에 맞는 알고리즘 선택
20분	각 풀이의 반례 3개와 검증	최소 입력·최댓값·불가능 경우
10분	핵심 풀이를 말로 설명	복잡도와 불변식, 남은 한계

본문 예제를 기억해서 빨리 푼 결과는 독립 평가로 세지 않습니다. 첫회는 형식 연습으로 쓰고, 다음회부터는 해설을 보지 않은 문제를 같은 시간으로 푼다. 실제 시험이 정해지면 공식 제한시간과 허용 도구에 맞춥니다.

오답 기록은 다섯 줄로

- 놓친 조건: 음수가 가능한데 양수 원도를 사용했다.
- 가장 작은 반례: $[1, -1, 5]$, $K=5$ 에서 답 1을 놓친다.
- 원인: 왼쪽을 줄이면 합이 반드시 감소한다는 전제가 깨졌다.
- 수정 원리: 문제 조건에 맞는 누적합 기반 접근을 다시 설계한다.
- 재풀이일·도움 사용 여부·결과: 2일/7일 후 각각 기록한다.

각 문제를 조건 해석·정확성 설명·복잡도·구현·검증의 다섯 항목으로 점검합니다. 해설을 보고 맞힌 것은 학습 완료, 해설 없이 다시 풀고 설명한 것은 재현 완료로 구별합니다. 기술 면접에서는 장비 업무의 큐·이벤트·경로 문제와 연결하되 실제 업무가 코테 알고리즘과 동일하다고 과장하지 않습니다.

검증 범위

문서의 작은 수치 예제와 알고리즘 결과는 별도로 검산했습니다. C++ 예제는 정적 검토한 학습용 코드이며 이 환경에서 컴파일·실행하지 않았습니다. 본인 환경에서 필요한 입력·main·테스트를 붙이고 경고를 켜서 빌드하세요.

APPENDIX / LOCAL NOTES

어떤 자료에서 이어졌나

경로는 표지의 기준 폴더 안에서 읽습니다. 기존 노트를 함께 펴놓고 이어서 학습하세요.

C++/1일차 - 배열·vector와 문자열·string.md

배열 순회·문자열·구간 합 연습을 다룬다. 여러 질의의 누적합과 갱신 차분으로 확장한다.

C++/2일차 - 빈도 배열과 해시 테이블 원리.md

빈도와 해시 원리를 기반으로, 이전 누적합 출현 횟수를 세는 문제를 추가한다.

C++/3일차 - STL 해시 컨테이너와 Stack·Queue·Priority Queue.md

컨테이너 사용 다음으로 최소 힙의 거리 후보 관리와 오래된 후보 제거를 연습한다.

C++/4일차 - 정렬과 이분 탐색.md

정답 이분 탐색과 LIS까지 있다. 새 내용으로 반복하기보다 단조성 증명·경계값 검증을 추가한다.

C++/5일차 - DFS·BFS와 간단한 동적 계획법.md

그래프·격자·백트래킹·DP가 이미 있다. 가중 최단거리와 DSU를 잇고 DP 갱신 순서를 반례로 검사한다.

C++ 문법/01.string과-char.md

문자열·포인터·수명·const를 보완한다. 문법은 문제 풀이 중 실제 오류가 생긴 항목을 복습한다.

검토 기준

기존 자료의 주제와 실습을 검토해 다음 과제를 골랐습니다. 문서에서 확인되지 않는 내용은 자료상의 빈칸으로 해석하며, 개인의 실제 경험이나 숙련도를 단정하지 않습니다.

REFERENCES / PRIMARY SOURCES

참고 자료와 활용 범위

본문의 [번호]에 대응하는 공식 문서·원문입니다. 확인일: 2026.09.26.

[1] 프로그래머스 · 파괴되지 않은 건물 (92344)

사각형 일괄 갱신의 2차원 차분 확장 문제. 본문 작은 수치 예제는 별도의 자작 문제입니다.

<https://school.programmers.co.kr/learn/courses/30/lessons/92344>

[2] 프로그래머스 · 연속된 부분 수열의 합 (178870)

양수 수열의 정확한 합, 최단 길이와 시작 인덱스 동률 처리. 본문 K 이상 함수와 조건이 다릅니다.

<https://school.programmers.co.kr/learn/courses/30/lessons/178870>

[3] 프로그래머스 · 배달 (12978)

무방향 가중 그래프·다중 간선·거리 상한 조건을 확인하고 Dijkstra 실습에 연결합니다.

<https://school.programmers.co.kr/learn/courses/30/lessons/12978>

[4] 프로그래머스 · 섬 연결하기 (42861)

전체 연결 비용 최소화 문제. DSU와 Kruskal을 적용하고 최단거리 문제와 구별합니다.

<https://school.programmers.co.kr/learn/courses/30/lessons/42861>